Signal iOS 8.14 (build 1637)

Reproducing the App Store build: a follow-up to BarbossHack's check

Prepared by commandersaki with Claude (Opus 5.5, Anthropic) in Claude Code · 7 November 2026

Summary. BarbossHack's automated check marked Signal iOS 8.14 as not reproducible, listing six files with "not acceptable differences". Using his own decrypted App Store IPA, the tag `8.14.0.1637` and the Xcode build Signal used (26.5, 17F42), **the compiled code (`__TEXT,__text`) of all 23 binaries in the app is byte-identical to the App Store build**: Signal's five own binaries and all 18 bundled frameworks, 65.5 MB of machine code in total. Each of the six flagged files is explained. `SignalUI` and `SignalServiceKit` differ only in the order of linker-generated GOT slots, and every affected stub and pointer still resolves to the same target. `Info.plist` differs only in build metadata (time, machine, build number). The two icons are rasterised at build time and differ by at most 8/255 per channel. `LibMobileCoin` has the same GOT-ordering effect: Telegram's `ipadiff.py` , the tool his workflow uses, flags it when the source is built with his exact command, and not in our archive build. None of the six reflects a difference in compiled code. The check fails because `ipadiff.py` doesn't normalise GOT order or Signal's build timestamp, and byte-compares rasterised icons.

1 Background

In post 17 of "Add reproducible builds to the iOS app" (15 June 2026), BarbossHack described an automated reproducibility check for Signal iOS:

1. He decrypted the App Store IPA with ipadecrypt on a jailbroken iPhone and published it in `BarbossHack/reproducible-LFS` .
2. A GitHub Actions workflow builds the release tag at `/Volumes/workspace/repository` with the Xcode named in `.xcode-version` .
3. Telegram's `tools/ipadiff.py` then compares the two IPAs.

For 8.14 (run 27575045209) the comparison failed with "not acceptable differences" in `SignalUI` , `LibMobileCoin` , `SignalServiceKit` , `Info.plist` , `AppIcon60x60@2x.png` and `AppIcon76x76@2x~ipad.png` . This report checks the same version with the harness from the 8.35.1 report, which accounts for every differing byte rather than giving a pass/fail verdict.

2 Inputs

Input	Value
App Store app	BarbossHack's `org.whispersystems.signal_8.14.decrypted.ipa` , SHA-256 `8a836442…d77e1d` . Version 8.14, build 1637. All 23 binaries have `cryptid 0` (decrypted).
Independent copy	The same version (App Store version ID `886450291`) downloaded separately with ipatool under a different Apple account and decrypted with unfairplay on a Mac. See below.
Source	Tag `8.14.0.1637` , commit `9a6ef4d` . The shipped `Info.plist` records `SignalCommit = 9a6ef4d…` , the same commit. Pods submodule `2f7bce7` ; libsignal `v0.94.1` , RingRTC `v2.69.1` , LibMobileCoin `signal/6.0.2` , all fetched by `make dependencies` at the versions the tag pins.

Input	Value
Toolchain	Xcode 26.5 (17F42), iOS SDK 26.5 (23F73), Metal toolchain 17F42. The shipped `Info.plist` records `DTXcodeBuild = 17F42` and `DTSDKBuild = 23F73`. Build machine: macOS 26.7 (25G229); Signal's was 25E246.

Two independent decryptions agree. The encrypted IPA we downloaded passes all 25,753 of its code-signature page hashes. Decryption changed nothing outside the FairPlay-encrypted ranges apart from the `cryptid` flag. All 349 files in the resulting app are byte-identical to the files in BarbossHack's IPA. Two different tools (ipadecrypt on an iPhone, unfairplay on a Mac), run by different people on downloads from different accounts, produced the same plaintext. As a separate check on his IPA alone, every code-signature page outside the encrypted ranges matches Apple's signature once `cryptid` is set back to 1 (0 mismatches across 23 binaries).

3 Build

The build is the same as for 8.35.1. The source is cloned to the Xcode Cloud path `/Volumes/workspace/repository` (on an APFS disk image named `workspace`), then built with `make dependencies` and an unsigned archive using the scheme's own "App Store Release" configuration:

```
xcodebuild archive -workspace Signal.xcworkspace -scheme Signal \
  -destination 'generic/platform=iOS' -derivedDataPath /Volumes/workspace/DerivedData \
  -archivePath /Volumes/workspace/build-814.xcarchive \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO CODE_SIGN_IDENTITY=""
```

BarbossHack's workflow differs in four ways:

- It uses `xcodebuild build`, not `archive`.
- It sets `CONFIGURATION_BUILD_DIR` and passes extra strip and coverage settings.
- It selects Xcode by the major.minor version alone (`check_xcode_version.py --relaxed`).
- It ad-hoc signs the result.

Section 4.6 shows what happens with his exact command.

4 Results

4.1 Compiled code: identical in all 23 binaries

`__TEXT,__text` was extracted with Apple's `segedit` and hashed with `shasum`. No script written for this project is involved in this step.

Binaries	`__text` bytes	Result
Signal, SignalServiceKit, SignalUI, SignalNSE, SignalShareExtension	31,894,628	✓ byte-identical (5/5)
18 bundled frameworks, including LibSignalClient, SignalRingRTC, WebRTC, LibMobileCoin, MobileCoin, SQLCipher, GRDB	33,612,020	✓ byte-identical (18/18)

WebRTC ships thinned. The build's copy has a fat wrapper containing a single arm64 slice, so that slice is extracted with `lipo -thin` before comparing, exactly as for 8.35.1.

The other layers of the harness agree. Across Signal's five own binaries, 144,393 of 144,393 functions (7.51 million instructions) match after address normalisation. Every string, ObjC/Swift metadata section, imported and exported symbol, linked library and section size is identical.

4.2 Every other differing byte

With code signatures removed, each binary was compared byte for byte, and every differing byte was assigned to a load-command field or section:

Difference	Where	Explanation
`__LINKEDIT.vmsize`, 1–2 bytes	All 23 binaries	The segment's size includes the code signature. Apple's signature is larger than an unsigned build's, so the rounded size differs. Nothing else in the header differs except the UUID.
`LC_UUID`, up to 16 bytes	12 of the 20 frameworks (blurhash, BonMot, GRDB, LibMobileCoin, LibSignalClient, Logging, Lottie, MobileCoin, SignalRingRTC, SignalServiceKit, SignalUI, SwiftProtobuf)	A build identifier set by the linker. Every other byte of these binaries is identical apart from the GOT ordering below, so the UUID must derive from link inputs that are later stripped, such as debug-map paths. It was not traced further. For 8.35.1 the UUID differed only in the main executable.
`__objc_stubs`, `__stubs`	SignalServiceKit (994 + 1 bytes), SignalUI (1,167 + 1)	The linker placed two equivalent GOT slots in the opposite order, so each stub loads its target from a different slot address. All 1,835 + 994 (SignalServiceKit) and 1,662 + 1,167 (SignalUI) stubs resolve to the same imported symbol or selector in both builds. For 8.35.1, building the same source twice showed this ordering is not deterministic.
`__TEXT,__const`, 12 bytes	SignalServiceKit	Twelve Swift relative pointers into the GOT. Each one, followed through the GOT in both builds, reaches the same target.

4.3 The six files BarbossHack's check flagged

File	Finding
`SignalUI`	✓ Code identical. Only the GOT slot order in the stubs differs (see above), along with the UUID and signature size.
`SignalServiceKit`	✓ Code identical. Same GOT ordering, plus 12 relative pointers into the GOT, all resolving to the same targets.
`LibMobileCoin`	✓ Code identical. In our archive build it differs only in `LC_UUID` and the `__LINKEDIT` size, which `ipadiff.py` ignores, so it reports equal. Built with his exact command, it shows the same GOT-ordering effect as `SignalUI` and `SignalServiceKit` (Section 4.6).
`Info.plist`	✓ The only differences are `BuildDetails.DateTime` and `Timestamp` (written by Signal's own build phase), `BuildMachineOSBuild`, `CFBundleVersion` (`1637` from Xcode Cloud versus `0` locally) and `DTAppStoreToolsBuild` (added by the App Store). `ipadiff.py` already ignores the last three. The 25 nested `Info.plist` files differ only in `BuildMachineOSBuild` and, for the two extensions, `CFBundleVersion`.

File	Finding
`AppIcon60x60@2x.png`, `AppIcon76x76@2x~ipad.png`	✓ Images with no executable content. The icons come from Icon Composer `.icon` files, which `actool` rasterises during the build. Decoded, 1,933 of 14,400 and 3,060 of 23,104 pixels differ, by at most 7/255 and 8/255 in one channel. The images look the same.

4.4 Asset catalogs

`ipadiff.py` skips `.car` files. Here they were compared rendition by rendition with Apple's `assetutil`:

- **`SignalNSE` catalog:** 8 of 8 renditions are identical.
- **`SignalShareExtension` catalog:** 2,370 of 2,378 are identical.
- **Main `Assets.car`:** 2,659 renditions are identical.

The differing ones fall into three groups:

- **Rasters of vector sources:** 13 renditions across the two larger catalogs. Each is a PDF asset rasterised by `actool`, and in every case the stored vector source has the same digest in both builds. Most are the 16-bit Display P3 variants.
- **Packed atlases:** 4. These are sheets of small rasterised images.
- **App icons:** the 72 rasterised "Icon Image" renditions. They exist in both catalogs with the same names, idioms, scales and sizes. All 290 other app-icon renditions are identical, including every vector layer, colour and gradient. The build also contains 24 1024-pixel marketing icons that the App Store strips.

Every differing rendition is a bitmap rasterised during the build from a source that is identical in both. The cause has been narrowed down but not proven. Xcode 26.5 renders these bitmaps inside an iOS Simulator agent (`AssetCatalogSimulatorAgent`). A later test on 8.15, which has the same assets, ruled out the Simulator runtime: re-running the logged `actool` step with the iOS 26.5 runtime matching Signal's SDK (23F73) and with the newer 23F77 gave the same differences (`versions/8.15.0.1655/out/asset_runtime_test.txt`). The remaining candidates are the GPU, since the agent draws with Metal on the host's graphics hardware and Signal builds on Xcode Cloud, or the host macOS version (25E246 versus 25G229). Neither could be tested here. The same test showed one vector raster (`clock-explorer`) varying between two runs on the same Mac. For 8.35.1, built with Xcode 27.1, whose renderer is different (`AssetCatalogAgent-AssetRuntime`), only the app icons differed, by at most 1/255, despite a larger macOS gap. Assets are images and colours with no executable content.

4.5 Telegram's `ipadiff.py` on this build

Running `ipadiff.py` (Telegram-iOS commit `9f4b352`) on our build reproduces BarbossHack's result. Our build was ad-hoc signed exactly as his workflow does. The tool reports `SignalUI`, `SignalServiceKit`, `Info.plist` and the two icons as different. The one exception is `LibMobileCoin`, which it reports as equal.

The tool explains these results. Its Mach-O comparison (`main.cpp`) zeroes the UUID, the `__LINKEDIT` sizes, the signature and the encryption offsets, and byte-compares everything else. Specifically:

- **`SignalUI` and `SignalServiceKit`:** the GOT slot order is not normalised, so whichever binaries the linker ordered differently are flagged.
- **`Info.plist`:** its plist rules ignore `BuildMachineOSBuild`, `CFBundleVersion` and `DTAppStoreToolsBuild`, but not the `BuildDetails` timestamp Signal adds.
- **The two icons:** PNGs are compared as raw bytes.
- **Skipped files:** it skips extensions (`PlugIns/`), asset catalogs and nibs entirely, so `SignalNSE` and `SignalShareExtension` are never checked.

The tool was written for Telegram's build and is strict in the ways that matter there. For Signal it reports differences that are layout and metadata, not code.

4.6 BarbossHack's exact build command

His run's logs and artifacts have expired (GitHub returns HTTP 410), so his build could not be inspected directly. Instead, the same source was built again on the same machine with his workflow's exact

`xcodebuild build` command. The result was ad-hoc signed and zipped exactly as his workflow does, then checked with `ipadiff.py`.

The check fails as his did, with six files flagged: `Info.plist`, the two icons, `LibMobileCoin`, `SignalServiceKit` and the main `Signal` executable. This time `Signal` is flagged instead of `SignalUI`. `__TEXT,__text` is byte-identical to the App Store's in all 23 binaries of this build too. In each flagged binary, the differing bytes are again GOT slot order:

- `Signal`: 1,669 + 1 stub bytes; all 4,077 + 1,669 stubs resolve to the same targets.
- `SignalServiceKit`: 15 relative pointers in `__const`, all reaching the same targets through the GOT.
- `LibMobileCoin`: 1 + 3 stub bytes; all 251 stubs resolve to the same targets.

Build of tag `8.14.0.1637`	**`Signal`**	**`SignalServiceKit`**	**`SignalUI`**	**`LibMobileCoin`**
BarbossHack's GitHub run	equal	flagged	flagged	flagged
This report: `xcodebuild archive`	equal	flagged	flagged	equal
This report: his exact `xcodebuild build`	flagged	flagged	equal	flagged

The binaries flagged change from build to build while the compiled code stays identical. Which ones `ipadiff.py` flags depends on how the linker happens to order equivalent GOT slots, not on what code is in the app.

5 Limitations

- **The public source itself is not reviewed.** This shows the App Store binary corresponds to tag `8.14.0.1637`. It does not show that the source is free of backdoors.
- **Prebuilt libraries: only libmobilecoin rebuilt.** This version's libmobilecoin (`signal/6.0.2`) is the same artifact that was rebuilt from source and matched byte for byte for 8.35.1 and 8.36 (8.35.1 report, section "Prebuilt native libraries rebuilt from source"). Its other prebuilt libraries, libsignal (`v0.94.1`), RingRTC (`v2.69.1`) and WebRTC (`7778b`), were checked against their pins only, not rebuilt. For the later versions 8.35.1 and 8.36, every prebuilt library reproduced exactly from public source with the projects' own recipes. That doesn't prove the same for these earlier builds, but it is a positive indicator that they too were built from their public source.
- **Some causes are explained but not proven.** The UUID and asset-rasterisation differences are attributed to build-environment details that were not reproduced (for the assets, the Simulator runtime has been ruled out; see Section 4.4). Two local builds of 8.14 produced different GOT slot orders, so that ordering varies between builds. Those two builds used different `xcodebuild` commands, though; the same-command test was done for 8.35.1. None of these differences touches compiled code, which is identical.
- **One version.** The result is for 8.14 (1637), and separately for 8.35.1 (1964). It says nothing about other releases.

6 Reproducing

Everything is in `commandersaki/signal-ios-build-verification`. This report and its outputs are in `versions/8.14.0.1637/`; BarbossHack's decrypted app is in `versions/8.14.0.1637/signal_dec/`. The scripts are the shared ones in `verify/`.

```
# 1. Source at the Xcode Cloud path, Xcode 26.5 (17F42) selected via DEVELOPER_DIR
git clone https://github.com/signalapp/Signal-iOS.git /Volumes/workspace/repository
cd /Volumes/workspace/repository && git checkout 8.14.0.1637 && make dependencies
# 2. Build: the xcodebuild archive command in section 3
# 3. Compare: functions/strings/symbols, then account for every other byte and file
APP=versions/8.14.0.1637/signal_dec/Payload/Signal.app
BUILT=/Volumes/workspace/build-814.xcarchive/Products/Applications/Signal.app
verify/compare_build.sh $APP $BUILT versions/8.14.0.1637/out/ipa_vs_build
verify/explain_residuals.sh $APP $BUILT versions/8.14.0.1637/out/residuals
```

Credit: BarbossHack for the decrypted IPAs and the original workflow; Telegram for `ipadiff.py`; DrWhax for unfairplay.

7 Changelog

Date	Change
7 November 2026	First published version, at `versions/8.14/`.
7 November 2026	**Moved** to `versions/8.14.0.1637/`; paths updated. **Changed** the cause given for the asset-rasterisation differences (Section 4.4): it previously named the macOS version as the likely cause. A test on 8.15 ruled out the Simulator runtime; the GPU or the host macOS remain. **Added** this changelog.
8 November 2026	**Changed** the prebuilt-libraries limitation: libmobilecoin is now verified (the same artifact was rebuilt from source for 8.35.1 and 8.36); libsignal, RingRTC and WebRTC for this version are not, though later versions reproducing exactly is a positive indicator.